

Homework



Homework

- In today's homework, we will practice more basic redux, but more specifically react redux.
- Before starting, let's revisit the concept of redux.
- Redux is a standalone tool that can also work with other applications besides React.
- So you just heard this: **redux** and then **react-redux**
- What? Does that mean there are 2 redux libraries? Yes, and here's the explanation:
 - Redux gives you a store, and lets you keep state in it, and get state out, and respond when the state changes. But that's all it does. It's actually react-redux that lets you connect pieces of the state to React components. So yes: redux knows nothing about React *at all*.
- These libraries are like two peas in a pod. 99.999% of the time, when anyone mentions "Redux" in the context of React, they are referring to both of these libraries in tandem. So keep that in mind when you see Redux mentioned on StackOverflow, or Reddit, or elsewhere. Like previous examples, this exercise illustrates redux *and* react-redux working together.



Homework

- Let's start by creating a new Github repository **m6-w2-d2-homework** and then clone this in your WESTCLIFF folder.
- Launch VS Code and navigate to this clone folder. After this, scaffold a react app
`npx create-react-app redux-counter`
- CD to the **redux-counter** folder.
- As a quick review, here's how our initial react counter works:
 - The count state is stored in the top level Counter component
 - When the user clicks "+", the button's onClick handler is called, which is bound to the increment function in the Counter component.
 - The increment function updates the state with the new count.
 - Because state was changed, React re-renders the Counter component (and its children), and the new counter value is displayed.



Homework

- For this app, we will only be using 2 files: index.js and Counter.js. For this we will skip App.js. The App component will be inside index.js.
- Open [index.js](#). Replace with this:

```
import React from 'react';
import { render } from 'react-dom';
import Counter from './Counter';

const App = () => (
  <div>
    <Counter />
  </div>
);

render(<App />, document.getElementById('root'));
```



Homework

- Now create **Counter.js** in the src folder, and enter the following codes:

```
import React from 'react';

class Counter extends React.Component {
  render() {
    return (
      <div>
        <h2>Counter</h2>
        <div>
          <span>{this.state.count}</span>
          <button onClick={this.decrement}>-</button>
          <button onClick={this.increment}>+</button>
        </div>
      </div>
    )
  }
}

export default Counter;
```



Homework

- Above `render()`, create the two handlers that the two buttons called, and also initialize count in the state to 0:
- Testing the app (**`npm start`**) at this time will work as expected.

```
class Counter extends React.Component {  
  state = { count: 0 }  
  
  increment = () => {  
    this.setState({  
      count: this.state.count + 1  
    });  
  }  
  
  decrement = () => {  
    this.setState({  
      count: this.state.count - 1  
    });  
  }  
  
  render() {
```



Homework

On to React Redux

- As discussed in the lectures, Redux keeps the **state** of your app in a single **store**. Then, you can extract parts of that state and plug it into your components as props. This lets you keep data in one global place (the store) and feed it directly to *any* component in the app, without the gymnastics of passing props down multiple levels.
- As a side note: you'll often see the words "state" and "store" used interchangeably. Technically, the **state** is the data, and the **store** is where it's kept.
- Next, let's add Redux to the project:

`npm i redux react-redux`



Homework

Last Things First

- Most app starts by creating a store, setting up Redux, writing a reducer, and so on. Lots must happen before anything appears on screen.
- We are going to take a backwards approach, and it will take just as much code to make things appear on screen, but hopefully the motivation behind each step will be clearer.
- So our counter app right now uses React component state to store data, which works. Let's just imagine for a second that we move the component's state into Redux.
- We'll remove the state from the component, since we'll be getting that from Redux soon:

```
class Counter extends React.Component {  
  
  increment = () => {  
    //fill in later  
  }  
  
  decrement = () => {  
    //fill in later  
  }  
  
  render() {
```

```
<span>{this.props.count}</span>  
<button onClick={this.decrement}>-</button>  
<button onClick={this.increment}>+</button>
```




Homework

Wiring Up The Counter

- Notice that `{this.state.count}` has been changed to `{this.props.count}`. This won't work yet, of course, because the Counter is not receiving a count prop. We're going to use Redux to inject that.
- To get the count out of Redux, we first need to import the connect function at the top:

```
import React from 'react';  
import { connect } from 'react-redux';
```

- Then we need to “connect” the Counter component to Redux at the bottom:

```
// Then replace this:  
// export default Counter;  
  
// With this:  
export default connect()(Counter);
```

- Where previously we were exporting the component itself, now we're wrapping it with this connect function call.



Homework

What's connect?

- You might notice the call looks a little... strange. Why **connect()(Counter)** and not **connect (Counter)**.
- It's written this way because connect is a *higher-order function*, which is a fancy way of saying it returns a function when you call it. And then calling *that* function with a component returns a new (wrapped) component.
- Next, add the `mapStateToProps()` function to return the state data from Redux. Create this between the Counter component and the export default:

```
}  
  
// Add the mapStateToProps function:  
function mapStateToProps(state) {  
  return {  
    count: state.count  
  };  
}  
  
// Then replace this:  
// export default Counter;
```



Homework

- And pass the **mapStateToProps** function to the connect function:

```
// With this:  
export default connect(mapStateToProps)(Counter);
```

- What connect does is hook into Redux, pull out the entire state, and pass it through the mapStateToProps function that you provide. This needs to be a custom function because only *you* will know the “shape” of the state in Redux.
- connect passes the entire state as if to say, “Hey, tell me what you need out of this jumbled mess.”
- The object you return from mapStateToProps gets fed into your component as props. The example above will pass state.count as the value of the count prop: the keys in the object become prop names, and their corresponding values become the props’ values. So you see, this function literally *defines a mapping from state into props*.



Homework

- At this time, you will get an error when testing the app.
- Since connect pulls data from the Redux store, and we haven't set up a store or told the app how to find it, this would be logical that it doesn't work. Redux has no idea what's going on right now.

Provide a Store

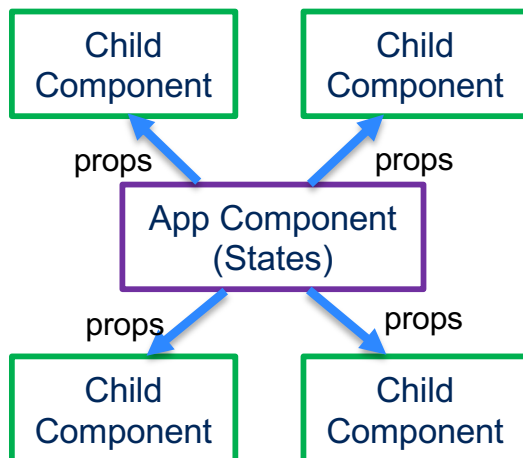
- Redux holds the global state for the entire app, and by wrapping the entire app with the Provider component from react-redux, *every component* in the app tree will be able to use connect to access the Redux store if it wants to.
- This means App, and children of App (like Counter), and children of their children, and so on – all of them can now access the Redux store, but only if they are explicitly wrapped by a call to connect.
- We are not saying to actually do that – connecting every single component would be a bad idea (messy design, and slow too).



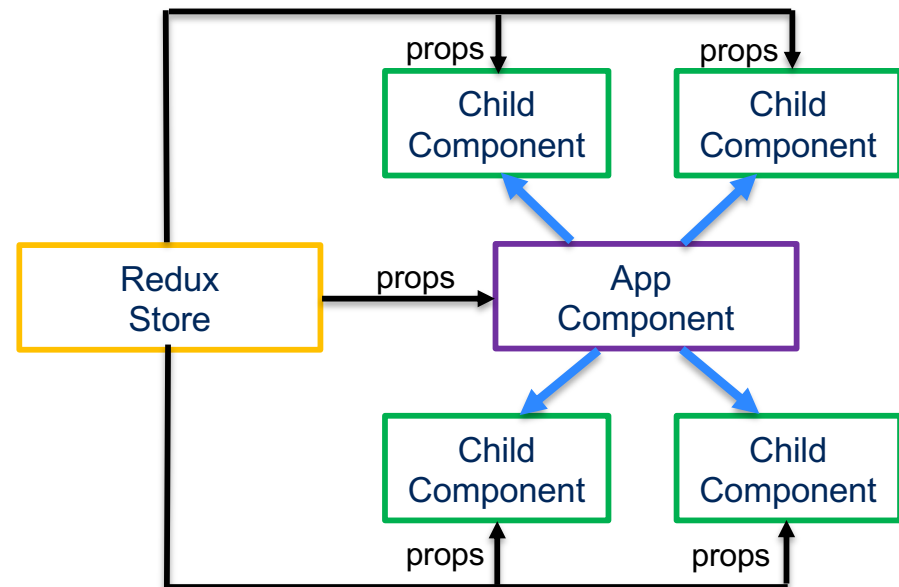
Homework

- To help visualize this:

React Component passing States to child components



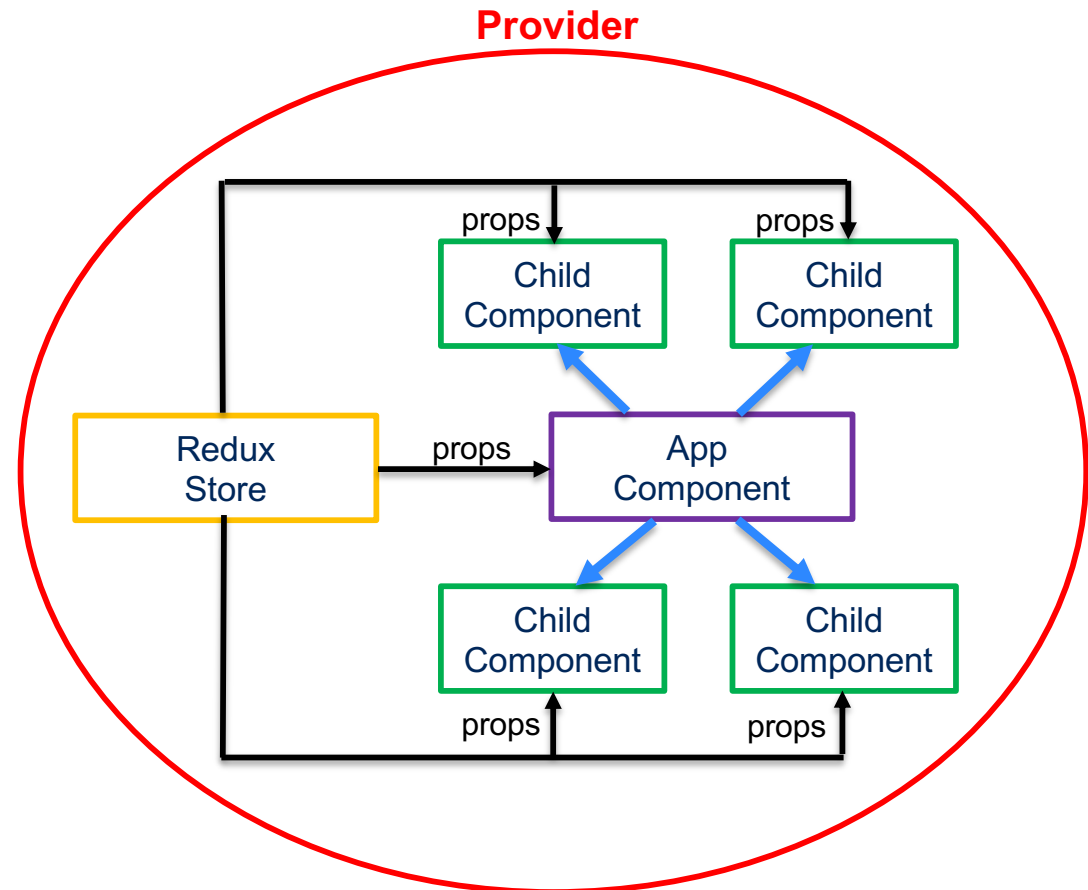
Redux passing states to all components directly





Homework

- This Provider thing might seem like total magic right now. It is a little bit; it actually uses React's "context" feature under the hood.
- It's like a secret passageway connected to every component, and using connect opens the door to the passageway.
- Imagine pouring syrup on a pile of pancakes, and how it manages to make its way into ALL the pancakes even though you just poured it on the top one. Provider does that for Redux.





Homework

- In src/index.js, import the Provider and wrap the contents of App with it:

```
import Counter from './Counter';  
import { Provider } from 'react-redux';  
  
const App = () => (  
  <Provider>  
    <Counter/>  
  </Provider>  
)
```

- At this time, we're still getting error – that's because Provider needs a store to work with. It'll take the store as a prop, but we need to create one first.



Homework

Create the Store

- Redux comes with a handy function that creates stores, and it's called createStore. Let's make a store and pass it to Provider:

```
import { Provider } from 'react-redux';  
import { createStore } from 'redux';  
  
const store = createStore();  
  
const App = () => (  
  <Provider store={store}>  
    <Counter/>  
  </Provider>  
)
```




Homework

- You will still get an error, but this time about the reducer:

Error: Expected the root reducer to be a function. Instead, received: 'undefined'

- So, here's the thing about Redux: it's not very smart. You might expect that by creating a store, it would give you a nice default value for the state inside that store. Maybe an empty object?
- But no: Redux makes *zero* assumptions about the shape of your state. It's up to you! It could be an object, or a number, or a string, or whatever you need. So we have to provide a function that will return the state. That function is called a **reducer** (we'll see why in a minute). So let's make the simplest one possible, pass it into createStore, and see what happens:

```
import { createStore } from 'redux';

function reducer() {
  // we'll leave this blank for now
  // which is the same as `return undefined;`
}

const store = createStore(reducer);
```



Homework

The Reducer Should Always Return Something

- Testing it again, reveal a different error this time:

TypeError: Cannot read property 'count' of undefined

- It's breaking because we're trying to access `state.count`, but `state` is undefined. Redux expected our reducer function to return a value for state, except that it (implicitly) returned undefined. Things are rightfully broken.
- The reducer is expected to return the state. It's actually supposed to take the *current* state and return the *new* state, but that's okay; we'll come back to that later.
- Let's make the reducer return something that matches the shape we need: an object with a `count` property. Next slide....



Homework

```
function reducer() {  
  return {  
    count: 42  
  };  
}
```

Counter

42 - +

- It works! The count now appears as “42”. Just one thing though: the count is forever stuck at 42.



Homework

The Story So Far

Before we get into how to actually *update* the counter, let's look at what we've done up till now:

- We wrote a `mapStateToProps` function that does what the name says: transforms the Redux state into an object containing props.
- We connected the Redux store to our Counter component with the `connect` function from `react-redux`, using the `mapStateToProps` function to configure how the connection works.
- We created a reducer function to tell Redux what our state should look like.
- We used the ingeniously-named `createStore` function to create a store, and passed it the reducer.
- We wrapped our whole app in the `Provider` component that comes with `react-redux`, and passed it our store as a prop.
- The app works flawlessly, except the fact that the counter is stuck at 42.



Homework

Interactivity (Making It Work)

- So far this is fairly simple and maybe even lame. You could've written a static HTML page with the number "42" and 2 broken buttons in 60 seconds flat, yet here you are, reading how to overcomplicate that very same thing with React and Redux and who knows what else.
- But the next section will make it all worthwhile.
- Or maybe not. A simple Counter app like this is a great learning tool, but Redux is absolutely overkill for something like this. React state is *perfectly fine* for something so simple. Heck, even plain JS would work great. Pick the right tool for the job. Just know that Redux is not always that tool.



Homework

Initial State

- So.. we need a way to tell Redux to change the counter.
- Remember the reducer function we wrote earlier? (of course you do, it was barely 2 minutes ago)
- Remember how was mentioned that it takes the *current state* and returns the *new state*? Well, not quite exactly. It actually takes the current state and an *action*, and then it returns the new state. We should have written it like this:

```
function reducer(state, action) {  
  return {  
    count: 42  
  };  
}
```

- The very first time Redux calls this function, it will pass undefined as the state. That is your cue to return the *initial state*. For us, that's probably an object with a count of 0.
- It's common to write the initial state above the reducer, and use ES6's default argument feature to provide a value for the state argument when it's undefined. See next slide...



Homework

```
import { createStore } from 'redux';

const initialState = {
  count: 0
};

function reducer(state = initialState, action) {
  return state;
}
```

- Try this out. It should still work, except now the counter is stuck at 0 instead of 42.



Homework

Action

- We're finally ready to talk about the action parameter. What is it? Where does it come from? How can we use it to change the counter?
- An “action” is a JS object that describes a change that we want to make. The only requirement is that the object needs to have a type property, and its value should be a string. Here's an example of an action:

```
{  
  type: "INCREMENT"  
}
```

```
{  
  type: "DECREMENT"  
}
```




Homework

Respond to Actions

- Remember the reducer's job is to take the *current state* and an *action* and figure out the new state. So if the reducer received an action like { type: "INCREMENT" }, what might you want to return as the new state?
- If you answered something like this, you're on the right track:

```
function reducer(state = initialState, action) {  
  if(action.type === "INCREMENT") {  
    return {  
      count: state.count + 1  
    };  
  }  
  
  return state;  
}
```



Homework

- Actually, it's more common to use a switch statement with cases for each action you want to handle. Change your reducer to look like this:

```
function reducer(state = initialState, action) {  
  switch(action.type) {  
    case 'INCREMENT':  
      return {  
        count: state.count + 1  
      };  
    case 'DECREMENT':  
      return {  
        count: state.count - 1  
      };  
    default:  
      return state;  
  }  
}
```



Homework

Always Return a State

- You'll notice that there's always the *fallback* case where all it does is return state. This is important, because Redux can (will) call your reducer with actions that it doesn't know what to do with. In fact, the very first action you'll receive is { type: "@@redux/INIT" }. Try putting a `console.log(action)` above the switch and see.
- Remember that the reducer's job is to return a *new state*, even if that state is unchanged from the current one. You never want to go from "having a state" to "state = undefined", right? That's what would happen if you left off the default case. Don't do that.



Homework

Never Change State

- One more thing to never do: do not *mutate* the state. State is immutable. You must never change it. That means you can't do this:

```
case 'INCREMENT':  
  // NO! BAD: this is changing state!  
  state.count++;  
  return state;  
  
case 'DECREMENT':  
  // NO! BAD: this is changing state too!  
  state.count--;  
  return state;  
  
default:  
  // this is fine.  
  return state;
```



Homework

Never Change State

- One more thing to never do: do not *mutate* the state. State is immutable. You must never change it. That means you can't do this:

You also can't do things like
`state.foo = 7`,
or `state.items.push(newItem)`
or `delete state.something`.

```
case 'INCREMENT':  
  // NO! BAD: this is changing state!  
  state.count++;  
  return state;  
  
case 'DECREMENT':  
  // NO! BAD: this is changing state too!  
  state.count--;  
  return state;  
  
default:  
  // this is fine.  
  return state;
```



Homework

All These Rules...

- Always return a state, never change state, don't connect every component, eat your broccoli, don't stay out past 11... it's exhausting. It's like a rules factory, and no one even know what that is.
- Yeah, Redux can be like an overbearing parent. But it comes from a place of love. Functional programming love.
- Redux is built on the idea of immutability, because mutating global state is the road to ruin.
- Have you ever kept a global object and used it to pass state around an app? It works great at first. Nice and easy. And then the state starts changing in unpredictable ways and it becomes impossible to find the code that's changing it.
- Redux avoids these problems with some simple rules. State is read-only, and actions are the only way to modify it. Changes happen one way, and one way only: action -> reducer -> new state. The reducer function must be "pure" – it cannot modify its arguments.
- There are even addon packages that let you log every action that comes through, rewind and replay them, and anything else you could imagine. Time-travel debugging was one of the original motivations for creating Redux.



Homework

Where Do Actions Come From?

- One piece of this puzzle remains: we need a way to feed an action into our reducer function so that we can increment and decrement the counter.
- Actions are not born, but they *are* **dispatched**, with a handy function called `dispatch`.
- The `dispatch` function is provided by the instance of the Redux store. That is to say, you can't just import `{ dispatch }` and be on your way. You can call `store.dispatch(someAction)`, but that's not very convenient since the store instance is only available in one file.
- As luck would have it, the `connect` function has our back. In addition to injecting the result of `mapStateToProps` as props, `connect` *also* injects the `dispatch` function as a prop. And with that bit of knowledge, we can finally get the counter working again.
- Here is the final component in all its glory. The only things that changed are the implementations of `increment` and `decrement`: they now call the `dispatch` prop, passing it an action.



Homework

- Let's return to Counter.js. Remember we left our handlers blank to be filled later. Well, now it's time to fill in the information. Actually, the actions dispatching that we had just talked about.
- Are all actions to be dispatched supposed to go here? No. They can be elsewhere as separate variables or even components, like in its own js file. In this example, we will dispatch directly in the Counter component, specifically in the handler that is called from the button:

```
class Counter extends React.Component {  
  
  increment = () => {  
    this.props.dispatch({ type: 'INCREMENT' });  
  }  
  
  decrement = () => {  
    this.props.dispatch({ type: 'DECREMENT' });  
  }  
  
  render() {
```




Homework

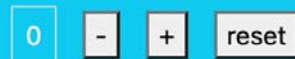
- The counter should work now. But let's give it a bit of styling. Install bootstrap, and then start styling. This is what it should look like:

Counter



- If you get to this point, good job! But wait, you're not done yet. Here's one more task for you, but a real simple one. You could possibly even close your eyes to do this. So here goes...

Counter



...and of course it must also work!



Homework

DUE:

On or before this Sunday, 10.30PM PT.

Submission:

Post GitHub URL on GAP Week 6 Day 2 Homework



Connect with Us (WEB503/803)

Professor

Rich Loke

richloke@westcliff.edu

Teaching
Assistant

Christopher Paul

christopherpaul@westcliff.edu

Teaching
Assistant

Ilya Valasov

ilyavalasov@westcliff.edu

Teaching
Assistant

Elizabeth Kipp

elizabethkipp@westcliff.edu

End of Presentation